AKSHAYA SAHAYI · DESIGN AND CODE REVIEW

Multiple organizations, customer portal, centre bookings and the platform-admin page

Everything built in this session, explained with diagrams and examples, so you can review it before it is committed.

Prepared 11 November 2026 · Branch `staging` · **Status: not committed, not deployed**
Code: 36 existing files changed (+576 / −167 lines), 25 new files · Tests: 4 automated test runs, all passing (details in section 14)

Contents

1. The problem and the decisions you made
2. The big picture: one database per organization
3. How a request finds the right database
4. Logins and tokens
5. Live updates (sockets), background jobs and links
6. Uploaded files
7. Portal links for each business
8. WhatsApp per organization and per centre
9. Booking with a specific centre
10. The platform-admin page
11. Plans, billing and automatic suspension
12. Database changes
13. Files changed: where to look
14. What was tested, and what was not
15. Switching it on at your VPS
16. Known gaps and next steps

Summary in one page

- **Each business that buys a subscription is an "organization" with its own PostgreSQL database.** Your current database becomes the *default* organization. No data moves.
- **Route code barely changed.** Every query still uses `pool.query(...)`. A new layer decides, for each request, which organization's database that goes to.
- **Each organization gets its own portal link** such as `/portal/buyer-x/login`, used by its staff and customers.
- **WhatsApp settings move from `.env` into each organization's Communication Settings.** Customer OTPs are sent from your platform number by default. Each centre has its own WhatsApp number or none.
- **Customers can book with a specific centre**, through a dropdown or the centre's own QR code. The superadmin picks *shared with confirmation* or *strict*.
- **A platform-admin page at `/platform`** lets you, the software owner, manage plans, organizations, trials, payments and suspensions. Plan limits are enforced inside the app.
- **It is safe to deploy before switching it on.** Without `CONTROL_DATABASE_URL` in `.env`, the server runs exactly as it does today.

1. The problem and the decisions you made

Before this work, the app could hold only one business:

- A superadmin was just a staff role, and `GET /api/centres` returned **every** centre to **any** superadmin.
- All customers were in one table, with Aadhaar unique across the whole system.
- Online service requests (`customer_services`) were one shared pool, seen by every centre.

If a second company had signed up, its superadmin would have seen your centres, customers and requests.

Decisions you made

Question	Your decision
How to separate businesses	**A separate database for each organization** (not one shared database with an organization column)
Same person registered with two businesses	Separate customer accounts per organization; documents never shared
How customers reach the right business	A path in the link: `yoursite.com/portal/<slug>`
Services catalogue	Each organization manages its own
Who creates organizations	You: a script first, then the platform-admin page
Hosting	Your own VPS, so extra databases cost nothing
Libromi developer API key	Different from the access token, so it gets its own field
Webhook secret check	Not needed: you register webhooks with Libromi by hand
Uploaded files	New files go under `uploads/<slug>/` ; old files stay where they are
Customer OTP sender	Your platform number by default; an organization may choose its own
WhatsApp per centre	Each centre its own number; no sharing; none means WhatsApp is off
Centre bookings	Centre links and QR codes, plus a superadmin choice of *shared with confirmation* or *strict*
Pricing	Per organization: tiered by centres and staff, or a flat fee; limits on centres, portal, WhatsApp and storage; monthly or yearly; trial and grace period; paid by hand for now, Razorpay later

2. The big picture: one database per organization

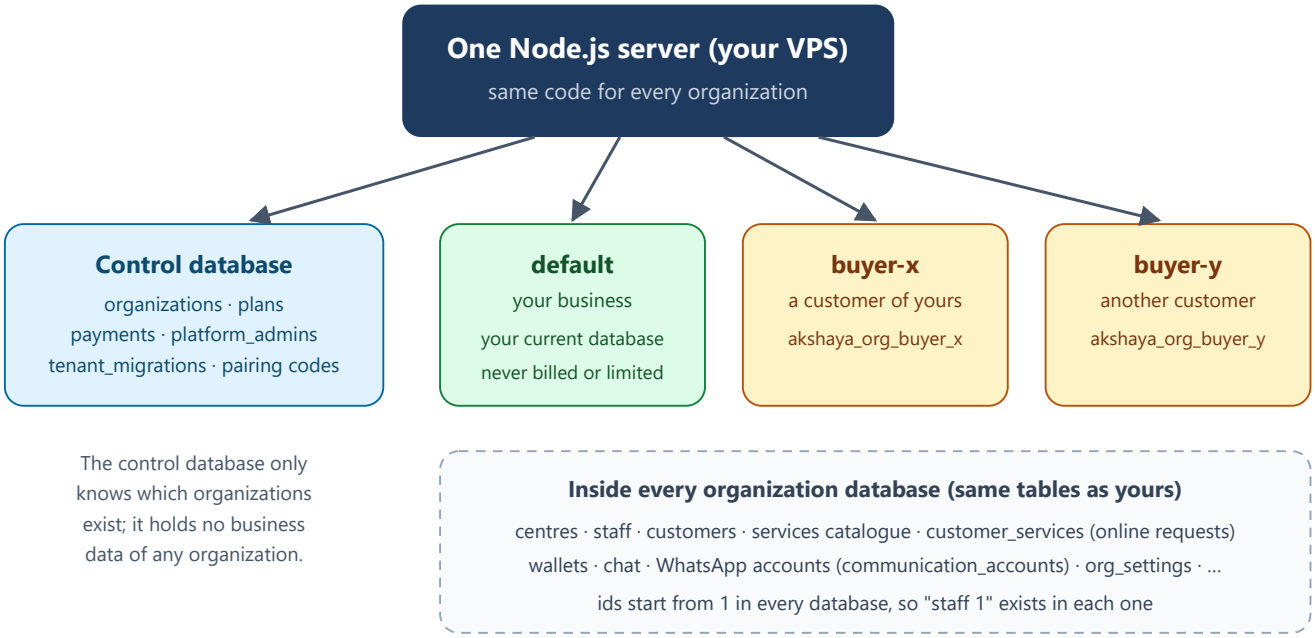


Figure 1. All organizations share one server and one Postgres instance, but each has its own database.

Why a database per organization

	Shared database + organization column	Database per organization (chosen)
Risk of leaking data	High: one forgotten `WHERE` shows another business's data	Very low: the data is physically in another database
Changes to your existing code	Every table and 50+ route files	Small: one layer under `pool`
Deleting or exporting one buyer	Hard	Drop or back up one database
Schema changes	Once	Once per organization, done by a script
Best for	Hundreds or thousands of small tenants	Tens of tenants (your case)

> **Single-organization mode stays available**
> Without `CONTROL_DATABASE_URL` in `.env`, everything runs on `DATABASE_URL` exactly as before. You can deploy the code first and switch multi-organization mode on later.

3. How a request finds the right database

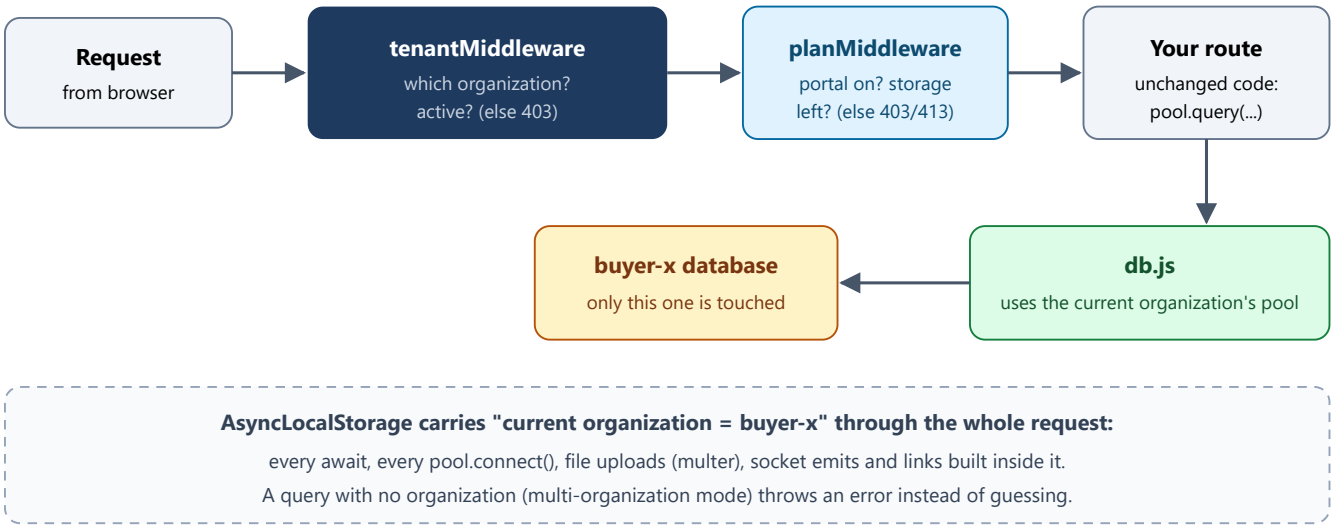


Figure 2. The path of one request. Only the two middlewares and db.js are new; route files keep their code.

Where the organization comes from, in order

#	Source	Used by	Example
1	The `org` claim in a signed login token	Logged-in staff, customers, companion phones	Token says `org: "buyer-x"`, so buyer-x is used, whatever else the request says
2	Print-station token `<slug>~secret`	The desktop print agent	`buyer-x~Zk3…`
3	`X-Org-Slug` header or `?org=`	Before login: login, register, OTP, public tracking; webhooks	`…/api/webhook/whatsapp?org=buyer-x`
4	Default	Everything else, including all your existing links	Your current site keeps working unchanged

The rule that prevents cross-organization access

A valid token always wins over headers. A token issued before this change has no `org` claim and belongs to the default organization. So a Buyer X user can't point their token at your database by adding `X-Org-Slug: default`. This was tested (section 14).

Example: the same staff id in two organizations

Both databases have a staff member with id 1: your `muhmdillyas`, and Buyer B's `bsuper`. Buyer B's superadmin uploaded a new profile photo (`PUT /api/staff/me`, a multipart upload):

- ✓ Buyer B's row 1 was updated and the photo saved under `/uploads/buyer-b/staff/…`
- ✓ Your row 1 was not touched
- ✓ A token claiming `org: "buyer-b"` with your user's id resolves against Buyer B's staff table, never yours

4. Logins and tokens

Login	Token contains	Notes
Staff (`/api/auth/login`)	`id, role, centre_id, org`	The response also returns `org_slug`
Customer (OTP)	`customer_id, org`	Signed with `CUSTOMER_JWT_SECRET`
Companion phone	pairing and device tokens carry `org`	The pairing token is in the request body, which the middleware also reads
Print agent	`<slug>~secret` for other organizations	6-digit pairing codes are registered in the control database so the agent reaches the right organization
Platform admin (you)	`type: "platform", pa_id`	Its own secret `PLATFORM_JWT_SECRET` . It never works on organization routes, and organization tokens never work on `/api/platform`

Example: three tokens, three outcomes

```
Staff token {id:1, role:"superadmin"}             → default organization (old-style token)
Staff token {id:1, role:"superadmin", org:"buyer-b"} → Buyer B's database
Platform token {type:"platform", pa_id:1}         → only /api/platform; 401 on /api/centres
```

5. Live updates (sockets), background jobs and links

Socket rooms are kept per organization

Room names like `user:1` or `centre:2` exist in every organization. The server now stores them as `<slug>|room`, and route code keeps calling `io.to(`user:${id}`)` unchanged.

```
Buyer B route:  io.to("user:1").emit("new_message")   →  room "buyer-b|user:1"
Your route:     io.to("user:1").emit("new_message")   →  room "default|user:1"
io.emit("tokenUpdate")  (used to go to everyone)      →  only "<current org>|*"
```

Background jobs run once per organization

All 8 cron jobs in `scheduler.js` and the print-file cleanup now run for each active organization in turn, each inside that organization's context. A failure in one organization is logged and doesn't stop the others.

Links sent to people

Review links sent by WhatsApp are built with `frontendUrl()`. For another organization they go through its portal path:

```
Your organization:  https://yoursite/review/abc123
Buyer X:            https://yoursite/portal/buyer-x/review/abc123
```

6. Uploaded files

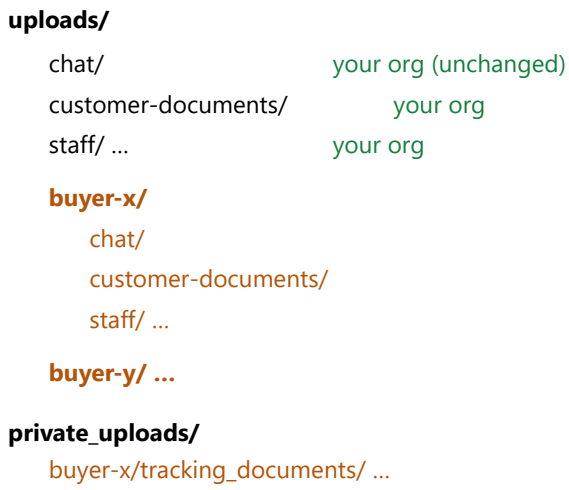


Figure 3. Your files stay where they are; each buyer gets one folder.

Why it matters

- **A buyer leaves:** their customers' documents are one folder to hand over or delete.
- **Backups:** one buyer's files can be backed up on their own.
- **Storage limits:** each buyer's usage is the size of their folder.

All 13 places that save or read files were changed (customer documents, photos, chat, knowledge, print jobs, tracking documents, centre logos, team files, staff photos). Database paths follow the folder, for example `/uploads/buyer-x/staff/staff-123.png`.

> **Rule for slugs**
>
> A slug can't be the name of an existing folder in `uploads/` (for example `chat`), or files would mix.

> **Still open (as before this work)**
>
> `/uploads` is served publicly. Anyone with the exact file link can open it. File names are hard to guess, but customer documents should later be served only through the logged-in download routes.

7. Portal links for each business

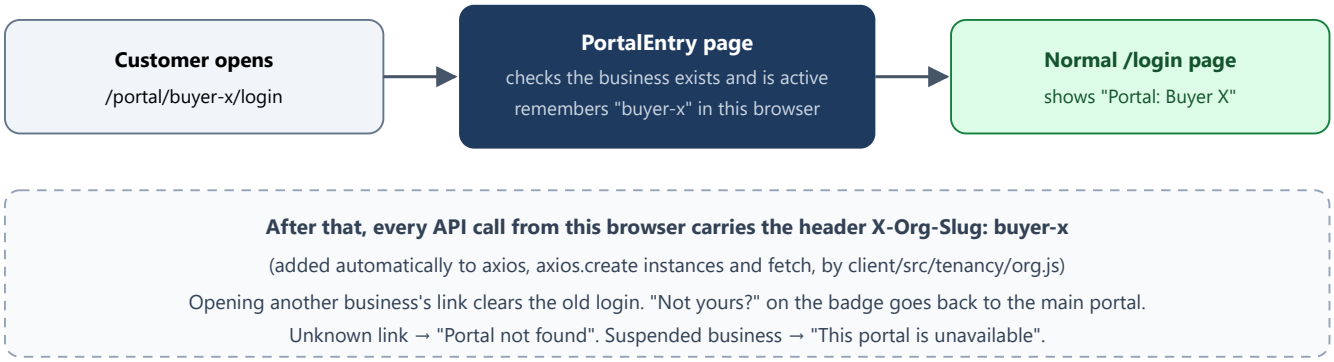


Figure 4. How a business's own portal link works.

Links each business gets

Link	Opens
`/portal/buyer-x/login`	Staff and customer login for Buyer X
`/portal/buyer-x/customer/register`	Customer registration
`/portal/buyer-x/c/3`	Centre 3's link or QR code: remembers the centre, then opens registration (or the dashboard if logged in)
`/portal/buyer-x/track/APP-…`	Public tracking page
`/login`, `/customer/register` …	Your own business, unchanged

8. WhatsApp per organization and per centre

Where the WhatsApp settings live now

Setting	Before	Now
Access token, channel id, number	Communication Settings (one database)	Communication Settings of each organization
Customer OTP sender	`LIBROMI_ACCESS_TOKEN` in `.env`	Your platform number (`PLATFORM_WHATSAPP_*` in `.env`), or the account the organization ticks
Developer API key (saves customer photos and documents)	`LIBROMI_API_KEY` in `.env`	A new field on each account; never shown again after saving
Webhook URL in Libromi	One fixed URL	Shown on the settings page; `…/api/webhook/whatsapp?org=<slug>` for other organizations
Bot / flow URLs (`/api/whatsapp/…`)	One business	Add `?org=<slug>` for other organizations
`LIBROMI_PHONE_NUMBER` , `CENTRE_PHONE`	In `.env`	Not used anywhere; can be removed

Which number sends a registration OTP

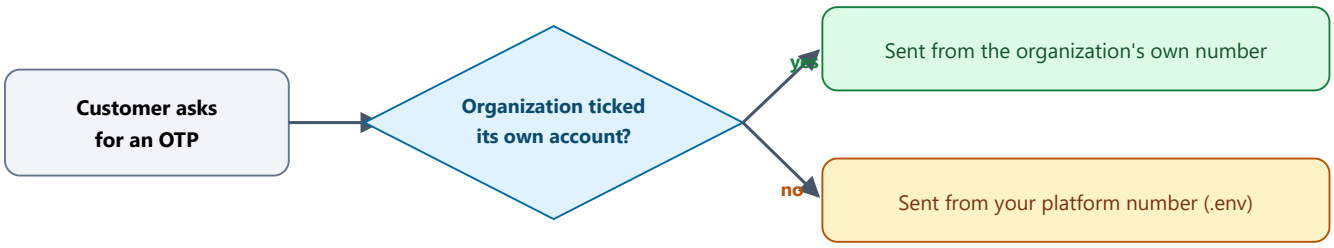


Figure 5. OTP sender. Organizations without WhatsApp can still register customers.

> **Cost and wording**
> Every OTP from the platform number is a Meta message you pay for, so build it into plan prices. Meta's OTP templates have fixed wording and show your platform's business name.

One WhatsApp number per centre

- Centre Management refuses a shared number: "*This WhatsApp number is already used by Centre A. Each centre needs its own WhatsApp number (or none).*"
- After the migration, the database blocks it too (a unique index).
- A centre without a number has WhatsApp off. Its chat button is hidden from customers.
- **Why:** an incoming WhatsApp message is matched to a centre by its number. With a shared number, the app couldn't tell which centre the customer meant.

Customer chat with a centre

The customer dashboard shows a **"Chat with a centre"** card listing every centre with WhatsApp connected, with the customer's own centre first. The button opens WhatsApp with a ready message, for example *"Hi Centre A, this is Anu."* Because the customer sends the first message, the centre can reply freely for 24 hours (a WhatsApp rule), and the message arrives in that centre's Messenger.

9. Booking with a specific centre

How the customer chooses

- The booking form has a **Centre** dropdown with an **"Any centre"** option.
- **Each centre's link or QR code** (for example `/portal/buyer-x/c/3`) pre-selects that centre. Superadmin → Settings → **Online Bookings** shows every centre's link, a copy button and a downloadable QR code to put at the counter.
- The choice is remembered for the next booking.

What is stored

Column (customer_services)	Meaning
`requested_centre_id`	The centre the customer chose. **Never changes**, so you always know what the customer really booked. Empty = any centre.
`handling_centre_id`	The centre of the staff member who took the job
`taken_from_other_centre`	True when another centre took it after confirming

The two modes (Settings → Online Bookings)

	Shared with confirmation (default)	Strict
Staff queue	Every booking, with a badge: Booked for Centre A (green for your centre, amber for others) or Any centre	Only your centre's bookings, plus "Any centre" bookings
Another centre clicks "Take Work"	Asked: "*This booking was made for Centre A. Take it anyway? Centre A will be told.*" If yes, it's taken, recorded, and Centre A's admin gets a notification	Refused
Checked by	The server (`PUT …/customer-services/:id/take`), not only the screen, so the rule can't be skipped	

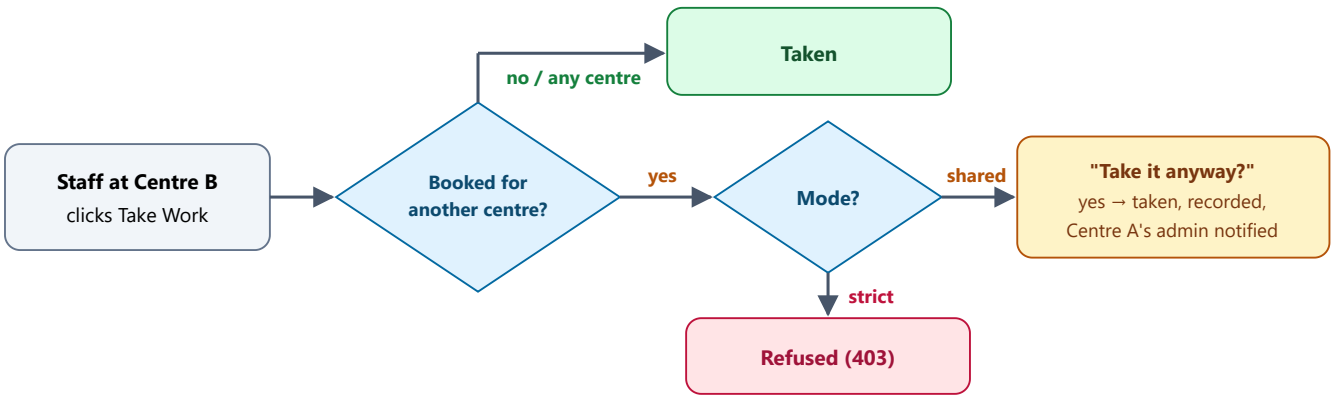


Figure 6. What happens when staff try to take a booking.

Example

Anu scans Centre A's QR code and books a passport service. In shared mode, Centre B's staff see it as Booked for Centre A, confirm, and take it. The booking keeps *requested* = *Centre A* and gets *handling* = *Centre*

B. Anu's "My services" shows Handled by Centre B. Centre A's admin is told: "*Staff B (Centre B) took booking APP-20261111-0004, which was made for Centre A.*"

10. The platform-admin page

A page for you, the software owner, at `/platform`. It has its own login and talks only to the control database, plus read-only usage counts from each organization.

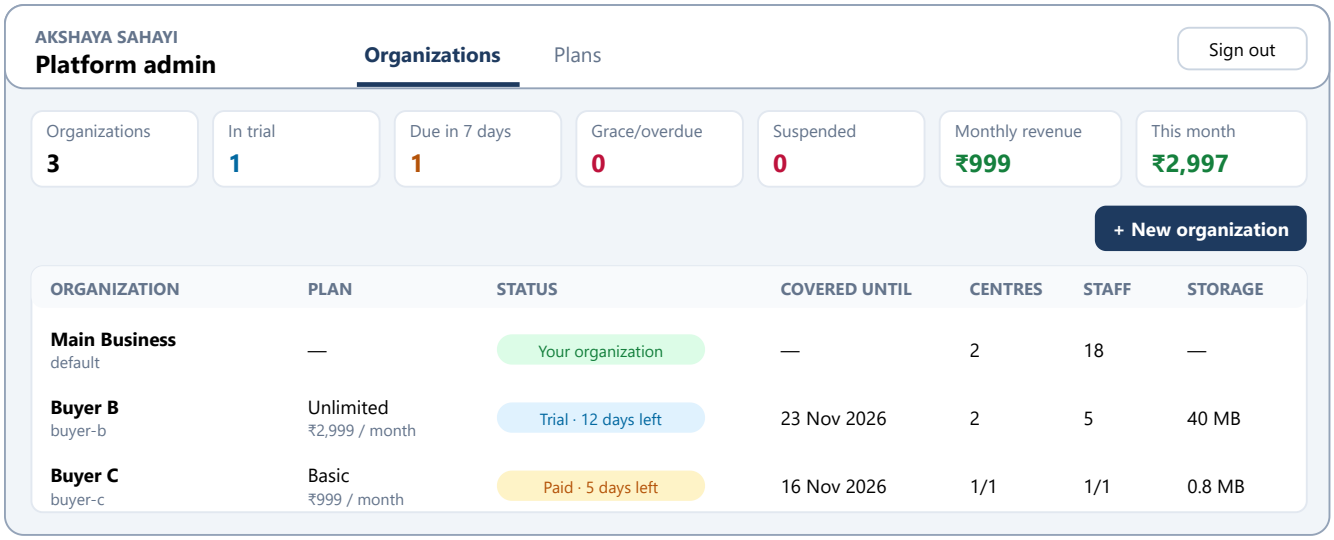


Figure 7. The Organizations tab (illustration with example data). Clicking a row opens its details.

What you can do

Screen	Actions
Plans	Create and edit plans: name, price per month and per year, limits on centres, staff and storage (empty = unlimited), customer portal on/off, WhatsApp on/off, available for new organizations
Organizations	Summary cards, list with filters (All, Needs attention, Trial, Suspended), and **New organization**: business name, portal address, their superadmin login, plan, billing cycle, free-trial days, grace days
One organization	Usage; edit details, plan and billing dates; **record a payment**; payment history; portal links with copy buttons; **suspend** or **switch back on**

Creating an organization from the page does the same as the script: it creates an empty database with your table structure, copies `document_master`, creates their superadmin, and registers it. It takes up to a minute.

11. Plans, billing and automatic suspension

Billing timeline



Figure 8. Example: Buyer X is created on 10 November with a 14-day trial and 7 grace days.

The rules

- An organization is covered until the **later** of its trial end and its paid-until date.
- After that it keeps working for its **grace days**. A daily job at **00:30 IST** then suspends it. You can also run the check from the API.
- **Recording a payment** adds one billing cycle after the current paid period (or from today if it had lapsed). You can also type an exact "covers until" date.
- A payment switches the organization back on **only if it was suspended for non-payment**. If you suspended it by hand, it stays suspended until you switch it on.
- An organization with no trial and no paid-until date is never suspended automatically.
- Your own organization is never billed, limited or suspended.

Payment examples (monthly plan, ₹999)

Situation	Payment recorded on	New "covered until"
Paid until 9 Dec 2026, paying early	10 Nov 2026	9 Jan 2027 (one month after 9 Dec)
Paid again straight away	10 Nov 2026	9 Feb 2027
Lapsed and auto-suspended	3 Dec 2026	2 Jan 2027, and switched back on

Plan limits inside the app

Limit	What the organization sees
Centres	Adding one more: "*Your plan allows 1 centres. Please contact us to upgrade your plan.*"
Staff	Same, for active staff members. Superadmins don't count.
Storage (MB)	Uploads refused once their folders reach the limit: "*Storage is full (… MB on your plan).*" Measured again after every upload.
Customer portal off	Customer registration, login and booking refused. Staff can still manage customers.
WhatsApp off	Accounts can't be added or switched on. Moving to such a plan switches existing accounts off. Chat buttons are hidden.

Plan changes, suspensions and payments take effect within 30 seconds (the organization list is cached for 30 seconds).

12. Database changes

Control database (new; created by `orgs.js init`)

Table	Holds
`organizations`	name, slug, database name, status and reason, plan, billing cycle, trial end, paid until, grace days, contact details, notes
`plans`	prices and limits
`payments`	payments you record: amount, date, period covered, method, reference
`platform_admins`	your platform-admin logins (hashed passwords)
`tenant_migrations`	which schema changes each organization's database has
`pairing_code_index`	live print-agent pairing codes and their organization

The server applies this schema itself at start-up (every statement is "if not exists"), so later additions need no manual step.

Changes to organization databases (run once per organization by the script)

File in `server/migrations/`	Change
`2026-11-10-whatsapp-per-org.sql`	`communication_accounts` : new columns `api_key` and `use_for_portal_otp`
`2026-11-11-centre-bookings.sql`	`customer_services` : `requested_centre_id` , `handling_centre_id` , `taken_from_other_centre` ; new table `org_settings`
`2026-11-11-one-whatsapp-per-centre.sql`	Stops and names the centres if any share a WhatsApp account; otherwise adds a unique index

> **From now on, every schema change goes through the script**
> Each organization has its own copy of the tables. Put the change in a `.sql` file and run `node scripts/orgs.js migrate <file>` . It applies the change to every organization and skips those that already have it. It also works before multi-organization mode is on.

13. Files changed: where to look

Ordered by how carefully to review them.

File	What changed
Core: the most important to review	
`server/tenancy/middleware.js`	Decides the organization of each request (new)
`server/db.js`	`pool` now uses the current organization's database
`server/tenancy/registry.js` , `context.js`	Organization list, connection pools, request context (new)
`server/tenancy/io.js` , `server/server.js`	Socket rooms per organization; wiring and start-up checks
Logins	
`routes/auth.js` , `customer.js` , `companion.js` , `printAgent.js`	Tokens carry the organization; print pairing across organizations
Platform admin (new)	
`routes/platform.js`	API: login, plans, organizations, payments, billing check
`tenancy/billing.js` , `limits.js` , `provision.js`	Billing rules and auto-suspend; plan limits; creating organizations
`scripts/orgs.js` , `tenancy/control_schema.sql`	Command-line tool; control database tables
`client/src/pages/platform/*`	The /platform pages
Bookings and WhatsApp	
`routes/servicemanagement.js` (take endpoint)	Shared and strict rules, takeover notification
`routes/customerBooking.js` , `staffPerformance.js`	Save the chosen centre; strict filter on the staff queue
`utils/whatsappAccount.js` , `whatsapp.js` , `routes/webhook.js`	OTP sender choice; developer key per account
`routes/communication.js` , `libromiWebhook.js` , `centres.js`	New settings fields; webhook URL per organization; one number per centre
`routes/orgSettings.js` , `portal.js` , `utils/orgSettings.js`	Booking mode setting; centre list for the portal (new)
`client/…/OnlineBookingSettings.jsx` , `ApplyForService.jsx` , `StaffDashboard.jsx` , `CentreChatCard.jsx` , `MyServices.jsx`	Settings tab with QR codes; centre dropdown; badges and confirm; chat card; "Handled by"
Portal links	
`client/src/tenancy/org.js` , `PortalEntry.jsx` , `OrgBadge.jsx` , `App.jsx` , `main.jsx`	Org header on every call; `/portal/<slug>` entry; "Portal: …" badge
Mechanical (lower risk)	
13 upload sites, `scheduler.js` , `printing.js` , `reviews.js` , `staff.js`	Upload paths per organization; jobs per organization; review links; staff limit

14. What was tested, and what was not

All tests ran against a copy of the server and temporary test databases (`zz_tenancy_test_*`), deleted afterwards. A fake Libromi server stood in for WhatsApp, so no real messages were sent.

Test run	Result	What it proved
Isolation between organizations	9/9	Logins stay in their organization; tokens can't be pointed elsewhere; same staff id in two organizations; uploads keep the context; suspension blocks
WhatsApp settings and uploads	11/11	Webhook URL per organization; OTP refused without a number (never the .env token for a buyer); developer key never returned; OTP choice; upload folders
Centre bookings and OTP number	20/20	Platform number by default; one number per centre (screen, migration, database); booking for a centre; shared and strict modes; customer sees both centres
Platform admin	35/35	Separate logins; plans; creating organizations; every limit; grace, overdue, auto-suspend; payments and reactivation; manual suspension kept; plan change switches WhatsApp off; summary
In the browser	Checked	Portal link login to Buyer B's dashboard; Online Bookings tab with QR codes; centre QR → customer dashboard with chat card; platform pages with data

Not verified

> **Your local database is older than production**
> It is missing tables such as `leads` , `scheduled_actions` and `print_jobs` , and some columns. Some screens and jobs could only be checked as far as that allows. Try these on staging:

- The notification to Centre A's admin when another centre takes a booking (the local `notifications` table is an old version)
- Real WhatsApp sending and receiving with real Libromi accounts, including the webhook with `?org=`
- Creating an organization from your production database's structure (`pg_dump` and `psql` must be installed on the VPS)
- The full staff dashboard and reports for a new organization

> **Already in your code before this work**
> `server/package.json` is missing packages the server imports: `pdfkit` , `exceljs` , `resend` , `pdf-lib` , `pdfjs-dist` , `officecrypto-tool` . They are presumably installed on the VPS, but a fresh install would fail.

15. Switching it on at your VPS

Do this on staging first. Steps 1–2 are safe on their own: without step 3 the server keeps running as one organization.

1. Deploy the code and apply the migrations

```
cd server
node scripts/orgs.js migrate migrations/2026-11-10-whatsapp-per-org.sql
node scripts/orgs.js migrate migrations/2026-11-11-centre-bookings.sql
node scripts/orgs.js migrate migrations/2026-11-11-one-whatsapp-per-centre.sql
```

If the last one stops and names centres sharing a WhatsApp account, give each centre its own account (or none) in Centre Management, then run it again.

2. Back up the database

```
pg_dump -Fc <your-database-name> > before-multi-org.dump
```

3. Switch on multiple organizations

```
# add to server/.env
CONTROL_DATABASE_URL=postgresql://USER:PASS@localhost:5432/akshaya_control
DEFAULT_ORG_SLUG=default
PLATFORM_JWT_SECRET=<a long random string>
PUBLIC_API_URL=https://api.akshayasahayi.com
# optional: a separate platform WhatsApp account for OTPs
# PLATFORM_WHATSAPP_ACCESS_TOKEN=…   PLATFORM_WHATSAPP_BASE_URL=…

node scripts/orgs.js init --name "Your business name"
node scripts/orgs.js add-admin --username you --password '<10+ characters>' --name "Your name"
# restart the server
```

4. Check

- Your staff and customers log in as before (no links change).
- `/platform/login` works; create your plans.
- Create a test organization from the page, open its `/portal/<slug>/login` link, and confirm it is empty and separate.

Going back

Remove `CONTROL_DATABASE_URL` from `.env` and restart. The server runs as a single organization again on your unchanged database.

16. Known gaps and next steps

Item	Notes
Online payments	Razorpay or another gateway; payments are recorded by hand for now
Due-date reminders	No messages to organizations before their trial or paid period ends
Customer documents served publicly	`/uploads` is public, as before; serve documents only through logged-in routes
Missing packages in `package.json`	Six packages the server imports aren't listed
Local development database	Older than production; refreshing it from a production schema dump would make local testing more complete
Storage of your own organization	Not measured (its files are mixed with the shared folder layout); it has no limits anyway
Strict mode and the queue counter	The staff queue and lists are filtered; other summary counts (for example reports) still count all bookings

Nothing is committed yet

After your review, the work can be committed to `staging` in one go, or split into separate commits (core, WhatsApp, bookings, platform admin) if you prefer.